

Write Up

Folder: PostgreSuperset

Filename: ollama_sql_ver2.py

- Uses class to store attribute of table schema, Here hourly_kpi table schema is hardcoded. All columns of table are given definition with the most suitable datatype and description
- Have added definitions to point out negative and positive metrics in concise manner
- Experimented on various ollama model along with hugging face model with '.gguf' support, not yet implemented model training, but that would require dataset to be generated. None the less, had found a synthetic dataset but haven't implemented model training. References:
 - https://huggingface.co/datasets/gretelai/synthetic_text_to_sql
 - <https://danielmiessler.com/blog/how-to-use-hugging-face-models-with-ollama>
- Flask's front html page of this script is mapped to index_ver2.html
- Which has wider viewport and functionalities to see ollama payload, sql query generated, data fetched from superset w.r.t. the sql. Shows exceptions that have aroused as well on a moderate level rather than checking up on terminal every time.
- Applying various models came to a conclusion that it doesn't provide the output in the mentioned format at least half of the time, thus have added a functionality to refractor the response generated by the llm to get the sql query and explanation if possible, using known common formatting mistakes.
- Few of the insights on the quality of results produced by these various models w.r.t. the questions are shared in the loop documentation on which task updates where made.
- The script the option to take command line arguments as port no. on which we want our flask application running and model on which we want to query our prompt in.
- Filename: ollama_sql_ver1.py is Eswar's implementation of the same problem.

Filename: analysis.py

- A cluster file to store all superset macros and functions, along with its respective similar functionalities from PostgreSQL access too.
- Some of the functions might not seem that much relevant, they are just reformatted version of Eswar's Sampling Correlation files which I had taken for reference. These include the files 'healthy_kpi.py', 'degraded_kpi.py', 'tmp.py' as well.

Folder: chromaDatabase

Filename: preprocess.py

- Instead of putting entire codebase together within 'ollama_pdf.py', I have separated some of the preprocessing functionalities into these files for simpler view of the code and modularity.
- All types of documentations are to be stored with 'documents' within this sub-directory. That's how the environment path is set by default. (\$DOCS_PATH)
- If the documents are of type 'docx', then it easier to preprocess by removing out unwanted titles and contents that we don't want like:
(titles_to_delete=["Contents, References, Annex"], remove_tables=True)
But by default, I have kept to ignore table of contents front page alone for now.
- Note: there is a problem when it comes to reading pdf files, as they are read page wise without much recognition to headings and blocks of text under each heading. (which are a very difficult task to do even with regex and font analysis as there just too many parameters that it could go wrong) Thus, after looking the pdf documentations of '*kpi_lte' and '*pm_lte', I have just added a simple regex format to remove TOC based on Contents pattern (title + dots + page number)
- For now, there are functionalities to read documentations of file extensions = ['.pdf', '.docx', '.xlsx', '.csv']. Note: '.doc' and '.docx' files aren't same and it won't be read by the database unless you implement functionality to convert it to docx and use existing method to read the converted file.
- In order to convert doc to docx, for Linux: subprocess launch LibreOffice convert with verbose off; for windows doc2docx python package will work. I have already added this function to convert all doc files in the folder specifies as parameter to be converted to docx, but not have used it explicitly anywhere for now.
- In overall its job is to process all documents in the folder specified inclusive all the sub-directories within it (os.walk). Finally, it returns the raw_text from each document along with its filename and its metadata.
- 'find_terms_and_abbreviations_in_sentence' in function methodology I have taken from preprocessing methods in Telco-RAG repository, as it could further improvise the prompt, but its current on TO DO status as we don't have a vocabulary files to extract only terms and abbreviations from. For now, its hardcoded as user_input if provided, it will be utilized to understand the question better by pipelining it through rag database collections.

Filename: ollama_pdf.py

- Using preheader.py file, in parent (root) directory for the custom logger formatting. This is just for convenience, it not necessarily needed.
- After processing the files, each file's raw_text is then splitted into chunks using existing langchain's text_splitter classes in case of document, whereas incase of excels or csv files, each row entry is taken as chunk as it makes no sense in split it too.
- If a filename has 'formula' in it, then it is stored into formula_db_collection, else based on extention, it gets stored into doc_db_collection or csv_db_collection.
- Similarity score used to measure embeddings is dot product here.
- The rag-pipeline has parameters to do only retrieval only and to enhance the user query or not. These param are directly linked to the attribute config['rag_generate'].
- Pipeline flow:
 1. enhances question using llm + term & abbreviations
 2. performs context retrieval on each collection as ordered in list as mention w.r.t. 'k_nearest' tuple variable of same size.
 3. Each retrieval task is one after another, adding the retrived content of all previous information onto next task's query as the way it is meant to be.
 4. Logs similarity score for query's embedding vs retrieved content's embeddings along with all the information w.r.t. chunk retrieved (including metadata, embedding size, filename, chunk_text)
 5. Finally performs response generation for the question based on the question and retrieved context as prompt well put into given 'chat_model' for llm.
- initialize_database() is function warped for modularity to re-implement this pipeline with further stages in other use case. Its parameters contain the necessary configurable parameters to use rag-pipeline as a retrieve-only block stage.
- Note: Apply refresh parameter only when documents existing inside the database are modified, it deleted the existing collection and reloads all info into chromaDB in setting to True. This parameter can be turn-off when a new file is added, as it just gets appended, but required when a document is removed to take effect into rag.
- This function initializes chroma database and client assuming that chroma is already setup in a separate ip address and port number, if not launch a local server in another terminal before running the code.
- Main function has the example implementation of how to use this pipelining provided some questions and answers (optional).

Folder: work (root)

Filename: preheader.py

- This is just a file I have used in common all over my files for custom logger to debug the code easier.
- All the code lines are out of function bounds, so in order to use this, you just have to simply add root to sys.path and import preheader.
- This file contains color coded custom formatter for logging and just for safety I have added 2 loggers to be running in background whenever this script is included.
 - The terminal logger is color coded, applying formatting to root logger ensures all file logger have the same properties as well.
 - Added logger with filestream output, so that whenever terminal overflows with log msgs and truncates previous log entries, it can be previewed completely in the filename.log file generated without the extension.

Filename: pipeline.py

- `refractor_response()` is the same fn that was existing in 'ollama_sql_ver2.py', here used again to parse the llm output to Json format to be able to get sql_query and explanation for in dictionary format as we needed.
- We will be executing the sql_query string in PostgreSQL instead of superset due to limitations of it.
- Pipeline goes as the question being sent rag context retrieval with the config parameters as we have previously seen in 'ollama_pdf.py' and then followed by a augmented prompt with table schema provided, with existing hardcoded table schema with attributes in 'ollama_sql_ver2.py' for now. (can be automated if all table schema are provided in sql attributed format [name, type, description] in yaml file properly for multiple table as it will our end goal)
- Rag configurable parameters:
`config = { 'embed_model', 'chat_model', 'chunk_size', 'chunk_overlap', 'k_nearest', 'rag_generate', 'sql_generate', 'terms', 'abbreviations' }`